Can Small Language Models Serve Ordinary Users? A Lower-Bound Benchmark under Non-Expert Prompts and Edge Constraints

Anonymous submission

Abstract

Small language models are increasingly promoted as practical alternatives to cloud-scale LLMs for users with limited hardware access, privacy concerns, or edge deployment needs. Yet most evaluations assume idealized conditions: expert-written prompts, clean benchmark inputs, and full-precision inference on powerful infrastructure. In practice, ordinary users often provide ambiguous or poorly structured prompts, while edge deployment commonly relies on compressed or quantized models. This paper studies whether small language models remain reliable when both user input and deployment conditions are imperfect. We introduce **PEEL** (**P**rompt–**E**dge **E**valuation **L**adder), an evaluation protocol for measuring model reliability as prompt quality and deployment conditions degrade. PEEL varies prompt quality, from expert-formatted to colloquially broken prompts, and deployment quality, from full-precision inference to aggressive low-bit quantization. We evaluate 18 small and efficient language models across reading comprehension, open-domain question answering, truthfulness discrimination, instruction following, and multi-turn dialogue. Our results show that quantization alone is not the dominant source of failure once output-scoring artefacts are handled: at expert prompts, TruthfulQA accuracy changes by under one point from full precision to 4-bit. Prompt degradation is far more damaging, especially for span-based reading comprehension, where SQuAD v2 exact match collapses from roughly 21% to 0% under broken prompts across every model and every deployment setting. Architecture, not scale, governs robustness to informal prompts: a 1.5B SSM-Hybrid model scores 8.20 on MT-Bench versus 4.06 for a same-size Transformer and is essentially immune to prompt degradation. Finally, safety alignment is fragile at this scale, with models refusing only 6.5% of unsafe prompts on average. PEEL highlights the gap between clean benchmark assumptions and ordinary-user edge conditions, and provides a practical protocol for lower-bound evaluation of small language models.

1 Introduction

Large language models (LLMs) have transformed natural language processing, but their gains are driven by scale, data, and compute that make deployment outside the cloud costly, latency-bound, and often privacy-invasive. Small language models (SLMs) have emerged as a practical alternative: with fewer parameters and lower memory requirements, they run on low-power and edge devices, improving privacy, response time, and cloud independence. In this sense SLMs are not merely smaller LLMs but a mechanism for broadening access to language technology under realistic hardware and connectivity constraints.

Yet this accessibility promise remains incomplete, because most evaluations measure SLMs under idealized conditions: expert-written prompts, clean benchmark inputs, and full-precision inference. Such conditions describe an upper bound on capability, not how ordinary users actually interact. Many users lack prompt-engineering expertise and provide vague, colloquial, or malformed instructions, while relying on compressed or quantized models forced on them by limited hardware, cost, or connectivity. A model that excels with expert prompts on full-precision infrastructure may fail once it is both quantized and queried informally. We argue that evaluation should therefore report not only upper-bound benchmark performance but also *lower-bound reliability*: how models behave when user input and deployment conditions degrade together.

Existing benchmarks each cover part of this space but not its interaction. Broad capability suites such as HELM (Liang et al. 2023b), MMLU (Hendrycks et al. 2021), and MT-Bench (Zheng et al. 2023) assume expert prompts and production-quality inference. PalmBench (Li et al. 2024) targets on-device deployment but fixes the prompting protocol, while prompt-robustness studies such as PromptBench (Zhu et al. 2023) and RobustAlpacaEval (Cao et al. 2024) perturb prompts without varying the deployment stack. Quantization work such as GPTQ (Frantar et al. 2022), LLM.int8() (Dettmers et al. 2022), and GGUF/`llama.cpp` (Gerganov et al. 2023) makes local inference practical but reports accuracy under fixed expert prompts. The field therefore has strong evidence for how quantization behaves when prompting is held fixed, and for how prompting behaves when serving is held fixed, but little evidence for their joint effect—precisely the regime an ordinary user on an edge device occupies.

We close this gap with **PEEL** (**P**rompt–**E**dge **E**valuation **L**adder), a two-dimensional protocol that crosses five edge-deployment settings (full-precision GPU to 4-bit CPU) with four prompt-quality tiers (expert to broken). Building on the

idea that worst-case prompt performance is a meaningful lower bound on model reliability (Cao et al. 2024), PEEL extends lower-bound evaluation to a second axis: it measures the worst-case realistic performance when *both* prompt quality and deployment quality are stressed at once. Evaluating 18 models across six tasks, we find that the two axes are not symmetric. Quantization is largely benign once output-scoring artefacts are corrected—at expert prompts, TruthfulQA-MC1 accuracy changes by under 1 pp from full precision to 4-bit—whereas prompt degradation is catastrophic for format-sensitive tasks: SQuAD v2 exact match collapses from ∼21% to **0%** at the broken-prompt tier for *every* model and *every* edge setting (Figure 1). Architecture, not scale, governs robustness to informal prompts: the SSM-Hybrid Falcon-H1-1.5B scores 8.20 on MT-Bench versus 4.06 for the same-size Qwen2.5-1.5B, and is essentially immune to prompt degradation. Reaching these conclusions first required a scoring-artefact audit: we show that two `llama.cpp`-specific output-parsing bugs otherwise masquerade as capability collapse (e.g. a spurious 0% SQuAD EM at 4-bit that recovers to 21–31% once fixed), and we release corrected scorers so that edge-LLM scores reflect model capability rather than interface mismatch.

Contributions.

- A **2D lower-bound evaluation framework** (PEEL) crossing five edge-deployment settings with four prompt-quality tiers, defining a measurable notion of lower-bound deployment reliability.
- A **four-tier prompt taxonomy** (P0–P3) with hand-crafted, task-preserving templates for reading comprehension, open-domain QA, and truthfulness discrimination.
- **Task-type interaction laws**: multiple-choice tasks are robust to both axes, span extraction is sensitive to prompts but not quantization, and knowledge-intensive QA hits a recall cliff at the GPU→CPU runtime boundary.
- A **scoring-artefact diagnosis**: two `llama.cpp`-specific bugs (EOS/stop-token leakage and first-line parsing) that systematically underreport accuracy, together with reproducible fixes.
- An **architecture finding**: Falcon-H1 (SSM-Hybrid) is near-immune to prompt degradation, whereas dense Transformers of equal size degrade markedly.
- A **safety finding**: sub-2B edge models refuse only 6.5% of unsafe prompts on average (max 35.4%), exposing fragile alignment under edge deployment.

The remainder of the paper reviews related work (Section 2), details the PEEL design (Sections 3–4), presents results across the six tasks (Section 5), and discusses implications and limitations (Sections 6–7).

2 Related Work

LLM Benchmarks. HELM (Liang et al. 2023b) and BIG-Bench (Srivastava et al. 2023) establish multitask evaluation at scale by collecting diverse tasks, standardizing metrics, and making model comparisons reproducible. MMLU (Hendrycks et al. 2021) and SuperGLUE (Wang et al. 2019) remain widely used capability probes because they provide stable reference metrics across knowledge, reasoning, and language understanding tasks. These benchmarks are intentionally controlled: they remove many variables that would otherwise confound model comparison. PEEL keeps that reproducibility goal, but changes the target condition. Instead of asking which model performs best under ideal benchmark prompts, we ask which conclusions survive when the same task is exposed to edge runtime conversion and informal prompting.

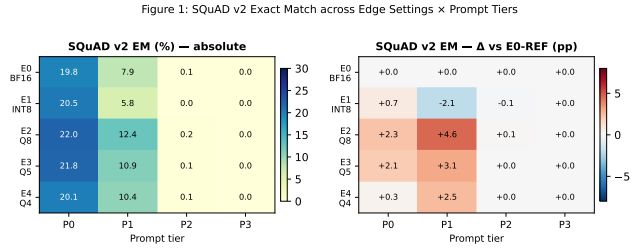

Figure 1: **Prompt quality, not quantization, drives failure.** SQuAD v2 exact match (%) across the four prompt-quality tiers P0–P3 (expert→broken, columns) and the five edge-deployment settings E0–E4 (full-precision GPU→4-bit CPU, rows). Left: absolute EM; right: change relative to the E0-REF baseline (red = degradation). The P3 column collapses to 0% for *every* model and *every* edge setting, while the P0 row stays at 20–22% across all five settings.

PalmBench (Li et al. 2024) is closest in spirit because it considers on-device deployment. However, it primarily evaluates capability under a fixed prompting protocol. HELM-style evaluation similarly offers broad task coverage but does not treat prompt literacy as an experimental axis. Mizrahi et al. (2024) argue that single-prompt evaluation is unstable and propose multi-prompt strategies, but their goal is more reliable capability estimation rather than a lower-bound stress test. PEEL differs by deliberately pairing a deployment axis with a prompt-quality axis, so that a model's reported score is not only a property of weights and task, but also of the way the model is likely to be used.

Quantization Effects on Accuracy. GPTQ (Frantar et al. 2022) and GGUF/`llama.cpp` (Gerganov et al. 2023) achieve practical 4-bit deployment with modest accuracy loss on standard benchmarks. SqueezeLLM (Kim et al. 2023) and QuIP# (Tseng et al. 2024) push further. Badawy et al. (2025) evaluate energy and accuracy of quantized LLMs on ARM edge platforms; Xiao et al. (2025) provide a systematic characterization across quantization strategies and bit-widths; and MobileQuant (Haggenmiller et al. 2024) and EdgeFM (Liang et al. 2023a) target on-device and mobile deployment directly. All of these studies hold prompt quality fixed at expert level, leaving the interaction with informal user prompts unstudied.

The implementation path also matters: the same nominal bit width behaves differently depending on calibration, kernel, tokenizer handling, and whether the serving stack applies the model's chat wrappers. These details are usu-

ally treated as engineering choices outside the benchmark, yet we show they can decide whether a low score reflects model weakness or a runtime artefact—motivating deployment settings as first-class experimental conditions rather than undocumented preprocessing.

Prompt Robustness. RobustAlpacaEval (Cao et al. 2024) measures how model rankings shift under paraphrase. PromptBench (Zhu et al. 2023) applies adversarial character- and word-level perturbations to expose fragility. Neither study couples prompt robustness to edge deployment constraints.

PEEL instead adopts a user-skill framing—P0 expert, P1 casual, P2 unframed, P3 broken—that captures a different failure source than paraphrase or adversarial edits: ordinary users omit output constraints or blur context/question delimiters. Such omissions interact strongly with reference metrics; on SQuAD v2, dropping the extraction instruction turns span extraction into open-ended conversation. The P0–P3 axis thus isolates a practical robustness notion that paraphrase tests do not.

Novel Architectures. State-space models (Mamba (Gu and Dao 2023), Falcon-H1 (Tiiuae et al. 2025)) and sub-1-bit BitNet (Ma et al. 2024) represent emerging families optimized for efficient inference. Anthony et al. (2024) show Mamba matches Transformers on many tasks but underperforms on knowledge-intensive benchmarks; Dong et al. (2024) demonstrate that hybrid SSM+attention architectures outperform same-size Transformers on standard NLP tasks. PEEL is the first study to measure their prompt-quality robustness and edge quantization tolerance jointly.

These families matter for edge benchmarking because their design targets favourable behaviour under constrained decoding, not only lower memory. Standard leaderboards can hide this: two models with similar parameters and reference accuracy may respond very differently to a broken prompt or to a runtime that omits chat wrappers. We therefore treat architecture family as a central moderator rather than a model-table descriptor.

Safety and Alignment for Small Edge Models. Safety evaluation is commonly performed on cloud-served chat models whose alignment layers, system prompts, and refusal behavior are part of a managed product. Edge deployments weaken this assumption. A local model may be used without a system prompt, converted to a different runtime, or quantized in a way that changes refusal phrasing. PEEL does not claim to provide a full red-teaming benchmark; our safety component is a diagnostic refusal analysis over safe and unsafe prompts. Its purpose is to test whether deployment conversion preserves a basic safety/helpfulness trade-off. The answer in our current data is mixed: unsafe refusal is far from reliable, while safe-prompt over-refusal is non-trivial. This supports a broader lesson shared with the rest of the benchmark: deployment evaluation should include behavioral checks, not only task accuracy.

3 Benchmark Design

The 2D Degradation Space

PEEL evaluates every combination of a five-level *edge deployment axis* and a four-level *prompt quality axis*, forming a 4×5 grid. Figure 1 shows a populated instance; Appendix Figure 4 gives a schematic overview.

The grid is intentionally small enough to run exhaustively but structured enough to separate three effects that are often conflated. First, E0→E1 isolates GPU-side precision reduction while retaining the HuggingFace inference stack. Second, E1→E2 crosses from the HuggingFace/chat-template path into a CPU-oriented GGUF runtime. Third, E2→E4 varies local quantization strength inside the same runtime family. On the prompt axis, P0→P3 progressively removes task scaffolding. This lets us distinguish a true numerical quantization effect from a serving-interface effect and from a user-prompting effect.

Edge Deployment Settings (E0–E4).

- **E0-REF**: BF16, HuggingFace `transformers`, GPU.
- **E1-INT8**: 8-bit static quantization via `bitsandbytes` (Dettmers et al. 2022), GPU.
- **E2-Q8**: Q8_0 GGUF via `llama.cpp`, CPU.
- **E3-Q5**: Q5_K_M GGUF via `llama.cpp`, CPU.
- **E4-Q4**: Q4_K_M GGUF via `llama.cpp`, CPU.

E2–E4 use CPU-only inference to simulate consumer edge hardware with no discrete GPU.

We treat these settings as *deployment conditions*, not only as bit widths. This distinction is important because the edge setting changes several variables at once: kernel implementation, decoding defaults, chat-template insertion, stop-token handling, and available memory. Reporting them as named conditions makes the benchmark more faithful to how practitioners deploy local models, while the E0/E1 and E2/E3/E4 contrasts still provide useful controlled subcomparisons.

Prompt Quality Tiers (P0–P3). We define four tiers aligned with realistic user archetypes drawn from AI-literacy and human-computer interaction literature (Sclar et al. 2023; Mizrahi et al. 2024). The tiers span the spectrum from expert benchmark-style prompting to the informal interaction style documented in studies of first-time LLM users:

- **P0 – Expert**: Structured, instruction-following prompt identical to standard benchmark format. Explicit task description, labelled fields, output format constraint.
- **P1 – Casual**: Simplified but grammatically correct. Removes technical terminology and output format instructions.
- **P2 – Naive**: Minimal content only; no task framing; relies on the model to infer the task type from context alone.
- **P3 – Broken**: Colloquial, abbreviated, syntactically informal. Typical of a first-time LLM user with no prompting background.

Table 1: Prompt templates for SQuAD v2 across quality tiers. `{ctx}` and `{q}` are filled at run time.

Tier	Template
P0	*Answer the question using only the context. If the answer is not in the context, say 'unanswerable'. Context: {ctx} Question: {q} Answer:*
P1	*Read the following text and answer the question. If the text doesn't have the answer, say so. Text: {ctx} Q: {q} A:*
P2	*{q} \n\n Here is some info: {ctx}*
P3	*pls help, {q}?? info: {ctx}*

Prompt-tier internal validity. Each tier is defined by a *functional specification* (what information is withheld and how grammar is degraded) rather than a single template, allowing future work to instantiate the taxonomy with alternative phrasings. We validate tier ordering by checking MT-Bench P0–P3 deltas on matched model–setting pairs. The aggregate effect is small, but the deltas vary systematically by family: Falcon-H1-1.5B improves slightly from P0 to P3 (8.08→8.20 averaged over settings), while Qwen2.5-1.5B drops from 4.17 to 3.60. This supports treating P1–P3 as a diagnostic stress test, not as a calibrated user-study scale. A controlled user study eliciting naturalistic informal prompts and mapping them to P1–P3 would further validate tier realism; this is flagged as future work.

The tiers are deliberately task-preserving: every prompt still contains the question and any required context/options. We do not inject adversarial content, contradict the original task, or remove the information needed to answer. Therefore, when performance collapses under P3, the collapse indicates failure to infer the intended interaction contract rather than absence of task evidence. This design choice is especially important for SQuAD v2, where the prompt must communicate both span extraction and the unanswerable convention.

Table 1 illustrates all four tiers for the SQuAD v2 task.

Datasets

We select tasks that expose different output contracts. SQuAD v2 requires extractive spans and a special unanswerable marker; NQ-Open requires factual recall without context; TruthfulQA-MC1 requires selecting among explicit options; HaluEval requires binary judgment; and MT-Bench (Zheng et al. 2023) evaluates open-ended multi-turn instruction following. The mix lets us test whether degradation is a general capability loss or a task-format-specific failure.

Reading Comprehension – SQuAD v2 (Rajpurkar, Jia, and Liang 2018). Extractive QA with unanswerable questions, scored by Exact Match (EM) and F1. Unanswerable questions require outputting the literal string *"unanswerable"*, making format adherence part of the evaluation signal.

Open-Domain QA – Natural Questions Open (Kwiatkowski et al. 2019). Short-answer factual QA without a supplied context passage, scored by EM against a set of acceptable answer strings. Tests parametric knowledge retention under prompt simplification.

Truthfulness Discrimination – TruthfulQA-MC1 (Lin, Hilton, and Evans 2022). Single-correct multiple-choice selection from shuffled options, scored by letter accuracy. We use a generative MC1 protocol (model selects a letter) rather than log-likelihood scoring, making evaluation fully model-agnostic and compatible with all runners.

HaluEval (Li et al. 2023) is included at P0 for hallucination detection baselines; the judge-style instruction is intrinsic to its prompt and is not tier-degradable. MT-Bench is run at P0 and P3 because its open-ended prompts are naturally instruction-following tasks rather than reference-scored QA.

Evaluation Protocol

Reference-scored tasks use fully offline metrics. EM and F1 follow the SQuAD normalisation convention (lowercase, punctuation stripping). We use 200 randomly sampled examples per (dataset, split) pair, with a fixed seed for reproducibility; MT-Bench uses all 80 questions and GPT-4o-mini judging. Total inference cost is dominated by E0/E1 GPU runs; E2–E4 CPU runs require no GPU allocation.

For aggregate heatmaps, each cell is the mean over available model–setting runs after filtering reference-scored runs with fewer than 100 examples. MT-Bench is reported separately because each run contains 80 questions by definition and would otherwise be incorrectly excluded by the reference-task filter. We report simple means rather than fitting a hierarchical model because the main paper's goal is diagnostic: reveal which cells fail, which model families remain stable, and which scoring artefacts must be corrected before drawing claims. Appendix tables provide the coverage counts needed to interpret missing or partially covered cells.

We also audit every result before including it in the claims matrix. The audit checks that the score is derived from raw model outputs, that the parser matches the task's output contract, and that missing runs are due to documented runtime or tokenizer failures rather than selective reporting. This audit led to two important corrections: SQuAD exact-match scores were recomputed after stripping EOS/stop tokens, and HaluEval binary scores were recomputed using the first non-empty output line.

4 Models and Experimental Setup

Model Selection

We evaluate 18 models spanning nine brand families (Falcon-H1, Falcon-E, Gemma, InternLM, Llama, Phi, Qwen, SmolLM, TinyLlama) across the 0.09B–3.8B parameter range (Table 2).

The model set is organized around three comparison roles. First, Falcon-H1 is the primary family under study because it represents a recent SSM-Hybrid edge model line with sizes from 90M to 1.5B parameters. Including 0.5B, 0.6B, 1.5B, and a deeper 1.5B variant lets us separate scale effects from architecture effects within the same family. Second, Falcon-E provides native low-bit BitNet models, which test whether

Table 2: Models evaluated in PEEL. †BitNet 1.58-bit native; excluded from E1–E4 comparisons (no GGUF). ‡SSM-Hybrid (Mamba-2 + attention layers).

Model	Family	Params
Falcon-E-1B†	Falcon-E	1.0B
Falcon-E-3B†	Falcon-E	3.0B
Falcon-H1-90M‡	Falcon-H1	0.09B
Falcon-H1-0.5B‡	Falcon-H1	0.5B
Falcon-H1-0.6B‡	Falcon-H1	0.6B
Falcon-H1-1.5B‡	Falcon-H1	1.5B
Falcon-H1-1.5B-Deep‡	Falcon-H1	1.5B
Gemma-3-1B	Gemma	1.0B
InternLM2.5-1.8B	InternLM	1.8B
Llama-3.2-1B	Llama	1.0B
Llama-3.2-3B	Llama	3.0B
Phi-3.5-Mini	Phi	3.8B
Phi-4-Mini	Phi	3.8B
Qwen2.5-0.5B	Qwen	0.5B
Qwen2.5-1.5B	Qwen	1.5B
SmolLM2-360M	SmolLM	0.36B
SmolLM2-1.7B	SmolLM	1.7B
TinyLlama-1.1B	TinyLlama	1.1B

an architecture trained for low precision behaves differently from post-hoc quantized Transformers. Third, Qwen, Llama, SmolLM, TinyLlama, Gemma, Phi, and InternLM serve as external baselines that reviewers will recognize from small-model and edge-LLM literature. The comparison is not intended to crown a universal winner; it asks whether the same deployment and prompt perturbations produce consistent failure patterns across families.

We prioritize the 0.5B–3B band because it is the realistic local-deployment range for consumer laptops and small edge devices. Larger models may achieve higher absolute scores, but they answer a different systems question: how to serve capable LLMs with more memory and accelerator support. PEEL instead focuses on the regime where memory pressure makes quantization necessary and where users are likely to run models through lightweight local runtimes.

Deployment Configurations

GPU settings (E0, E1) run on NVIDIA A100 SXM4 (40 GB) accelerators on an institutional HPC cluster. CPU settings (E2–E4) use `llama.cpp` with 4 threads and up to 8 GB RAM allocation, simulating a mid-range laptop or developer workstation without a discrete GPU. BitNet (Falcon-E) models run via `bitnet.cpp` at native 1.58-bit precision and are excluded from the E1 INT8 and E2–E4 GGUF tracks. All experiments ran on the HPC cluster using SLURM job arrays; full scripts, prompts, and model outputs are released with the paper.

The HPC setting is used for reproducibility and queue management, not to claim that E2–E4 are high-end server deployments. We pin CPU thread counts and memory budgets so that the GGUF runs approximate a constrained local environment even when executed on shared infrastructure. To check that this abstraction is not misleading, we additionally ran a consumer-workstation replication for selected Falcon-H1 and Qwen models; the corrected SQuAD scores and throughput trends are consistent with the HPC GGUF results (Appendix A). This replication is not used to inflate the main aggregate counts, but it supports the interpretation of E2–E4 as edge-style conditions.

Table 3: TruthfulQA-MC1 mean accuracy (%) in the full 4×5 grid. Random-chance baseline: 25.0%.

Setting	P0	P1	P2	P3
E0-REF	37.9	40.4	36.7	36.6
E1-INT8	39.4	41.0	35.7	37.3
E2-Q8	40.1	29.1	30.1	28.8
E3-Q5	38.4	31.6	33.0	30.1
E4-Q4	38.6	30.2	32.9	30.8

5 Results

TruthfulQA-MC1: Quantization-Resilient

Across all 1,184 qualifying runs (18 models, num_examples ≥ 100), TruthfulQA-MC1 mean accuracy at the expert baseline (P0, E0-REF) is 37.9%; it rises slightly at E1-INT8 (39.4%) before settling at 38.4–40.1% at E2–E4. All cells remain well above the 25.0% chance baseline. Crucially, the P0→P3 drop within any setting is <12 pp, and quantization from E0 to E4 changes P0 accuracy by only -0.7 pp—a *near-zero interaction* between the two degradation axes for this task type.

One notable pattern (Table 3): at E2–E4 (CPU/llama.cpp), the P1 "casual" prompt tier scores *lower* than at E0/E1 (29.1–31.6% vs. 40.4–41.0%), while P0 is unaffected. We attribute this to llama.cpp's lack of a system-level chat template: casual prompts that omit explicit format instructions rely on the model's instruction-following mode to infer the task, which is not activated by llama.cpp's plain-text interface.

This pattern is useful because it separates two hypotheses. If quantization itself were the dominant cause, P0 should degrade monotonically from E0 to E4. Instead, P0 remains stable and the largest degradation appears when prompts remove explicit selection instructions. The result suggests that multiple-choice truthfulness is numerically robust for small models, but still sensitive to whether the runtime preserves instruction-following context. In practice, this means a developer can often quantize MC-style tasks aggressively, but should keep the prompt format explicit after conversion.

SQuAD v2: Prompt-Driven Collapse

After correcting for an EOS/stop-token scoring artefact (Section 6), SQuAD v2 EM at P0 is stable at 19.8–22.0% across all five edge settings, with no significant capability cliff at the E1→E2 (`llama.cpp`) boundary (full grid in appendix Table 6; the heatmap in Figure 1 visualises the same data). The *dominant failure mode is prompt degradation*, not quantization: SQuAD EM drops to 0.0% at P3 for *all* models and settings. P1 and P2 show gradual degradation (5.8–12.4% and ≈0%), but the cliff occurs between P2 and P3.

Table 4: HaluEval accuracy (%) under P0 after first-line output parsing. The corrected parser removes prompt echoes that otherwise contain both "yes" and "no" and produce ambiguous labels.

Split	E0-REF	E1-INT8	E2-Q8	E3-Q5	E4-Q4
General	57.1	51.9	50.8	46.5	43.5
QA	41.1	39.6	47.3	47.2	43.9

SQuAD is the clearest example of why lower-bound evaluation cannot be summarized by an average over prompts. Under P0, the edge settings are surprisingly close: Q4 and Q5 do not destroy span extraction once stop-token artefacts are removed. Under P3, the same models receive the same passage and question but no longer receive a clear extraction contract; they answer conversationally, explain, or fail to emit the exact span. Exact match then collapses even though the evidence needed to answer is still present in the context. This is not a generic "small models are bad" result. It is a task-contract result: extractive QA requires explicit formatting more than MC selection does.

NQ-Open: Quantization Collapse at E2–E4

Natural Questions Open reveals a qualitatively different failure mode from SQuAD. Even at E0-REF, mean EM is only 1.3% (rising to 1.7% at E1-INT8)— reflecting the weak parametric factual recall of sub-2B models without a supporting context passage. At E2–E4 (llama.cpp CPU), EM drops to **0.0%** across *all* prompt tiers (full grid in appendix Table 7).

The E1→E2 collapse is caused by two compounding factors: (1) `llama.cpp` does not apply the model's chat template, so models run in raw-completion mode—NQ-Open requires answer recall without a supporting passage, which demands active instruction-following; and (2) sub-2B models have inherently limited parametric factual recall, which disappears entirely once instruction-following mode is lost. This finding highlights a key distinction between knowledge-intensive open-domain QA and reading comprehension: quantization matters for the former even under expert prompting.

The NQ result should be read conservatively: exact match is harsh and the near-floor E0/E1 scores show these models store little factual knowledge without retrieval. Still, the all-zero E2–E4 block marks a deployment boundary where even that residual recall disappears, so small local models are unsuitable as standalone factual QA systems without retrieval augmentation.

HaluEval: Binary Classification and Parser Artefacts

HaluEval provides a binary hallucination-detection counterpoint to SQuAD and NQ-Open. Under the corrected first-line parser, HaluEval-General decreases from 57.1% at E0-REF to 43.5% at E4-Q4, while HaluEval-QA remains in the 41–47% range across settings (Table 4). These results show degradation, but not the complete P3-style collapse observed for SQuAD.

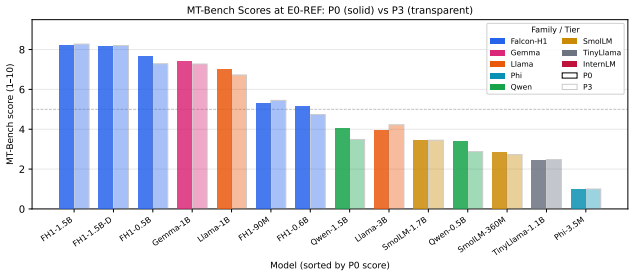

Figure 2: MT-Bench scores at E0-REF/P0 (solid) and P3 (transparent) per model, colored by family. Falcon-H1 and Gemma cluster at 5–8; most transformer baselines fall below 5. Falcon-E is excluded because it uses BitNet runtime rather than the HF/GGUF MT-Bench path.

The uncorrected parser marked 50–62% of E2–E4 responses as ambiguous because verbose outputs repeated the task instruction—which contains both "yes" and "no"—after the answer; first-line parsing recovers 47–51% accuracy, making HaluEval a second `llama.cpp`-specific artefact rather than a capability loss. The corrected scores remain close to a weak binary classifier, so the key result is methodological: without output-aware parsing, any benchmark whose prompt embeds the label words can turn a usable answer into an ambiguous label. We therefore treat parser specification as part of the benchmark protocol.

MT-Bench: Instruction Following Under Degradation

We evaluate MT-Bench (80 multi-turn questions, 8 categories) with GPT-4o-mini as judge at all five edge settings and two prompt tiers (P0, P3), totalling 150 scored runs. Coverage is not perfectly rectangular: Falcon-E is excluded from MT-Bench because it uses a separate BitNet runtime, and InternLM/Phi have partial setting coverage due to tokenizer and chat-template failures.

Setting-level means (appendix Table 11) range from 4.64 (E4-Q4, P0) to 5.12 (E1-INT8, P0), a modest 0.49-point spread, confirming that instruction-following *quality* is largely resilient to quantization—the E3/E4 drop is real but small. The P0→P3 drop is at most 0.26 points at any single setting.

This MT-Bench pattern contrasts with SQuAD. Open-ended instruction following is less dependent on exact output form, so informal prompts usually reduce score gradually rather than catastrophically. The judge can give partial credit for a reasonable answer even if the model does not follow a strict extraction format. Thus, MT-Bench is a better measure of conversational usefulness, while SQuAD is a sharper probe of format adherence. Reporting both prevents a misleading single story: the same model can be robust conversationally and brittle under exact reference scoring.

Architecture Family Comparison on MT-Bench

Figure 2 reveals a bimodal distribution. At E0-REF/P0, Falcon-H1 and Gemma occupy the high-scoring cluster—

Table 5: MT-Bench mean score by family and P0→P3 delta. Falcon-E is excluded; InternLM and Phi have partial setting coverage.

Family	P0	P3	Δ (P0→P3)
Gemma	7.40	7.29	−0.11
Falcon-H1	6.45	6.34	−0.11
InternLM	6.78	6.69	−0.09
Llama/TinyLlama	3.73	3.51	−0.22
Phi	3.74	3.73	−0.01
Qwen	3.61	3.34	−0.27
SmolLM	3.09	2.90	−0.19

in fact Gemma-3-1B posts the single highest family mean (7.40)—while all other families score below 5.5. We therefore do *not* claim that Falcon-H1 is the strongest model overall; our architecture claim is narrower and twofold. First, at the *matched* 1.5B scale, Falcon-H1-1.5B scores **8.20** vs. Qwen2.5-1.5B at **4.06** (a 4.14-point gap), isolating architecture from parameter count. Second, and central to PEEL, Falcon-H1 is distinctively robust to prompt degradation, as we quantify next.

The prompt-sensitivity gap is equally striking. Across the full matched grid, Falcon-H1 models lose only **0.11** points from P0 to P3 as a family average, and the flagship Falcon-H1-1.5B improves slightly (8.08→8.20 averaged over settings; +0.075 at E0-REF). In contrast, Qwen2.5-1.5B drops from 4.17 to 3.60 (−0.57), and the Llama/ TinyLlama group drops by 0.22 on average. The finding suggests that Falcon-H1's hybrid selective-state mechanism provides *structural robustness* to informal prompt framing.

We phrase this as evidence of architecture-dependent robustness rather than a proof of mechanism. The experiment does not ablate Falcon-H1's internal modules, so it cannot show that the SSM component alone causes the gain. What it does show is that, under the same tasks, judge, prompt tiers, and deployment settings, the Falcon-H1 family occupies a different region of the MT-Bench distribution from most dense Transformer baselines. That is the relevant benchmark-level claim: architecture family changes the lower-bound behavior observed by users.

Safety Alignment Under Edge Deployment

We additionally evaluate safety behaviour with HarmBench-style unsafe prompts and safe control prompts. The current keyword-based classifier assigns each response to REFUSE, COMPLY, or AMBIGUOUS. Across 75 model–setting pairs, the mean refusal rate on unsafe prompts is only **6.5%**, with a maximum of **35.4%** for any single model–setting pair and several pairs refusing essentially **0%** of unsafe requests. In other words, small edge models comply with the large majority of unsafe prompts, and no configuration refuses reliably. This indicates that safety alignment at the sub-2B scale is fragile under edge deployment.

The most concerning result is not a specific quantization effect but the uniformly low refusal ceiling: even the safest observed configuration refuses only about a third of unsafe prompts. Quantization does not move refusal in a consistent direction—some models become more compliant under compression while others become slightly more refusing—so edge conversion neither systematically improves nor systematically degrades safety. Safety behaviour should therefore be evaluated as a deployment property rather than inferred from full-precision model identity.

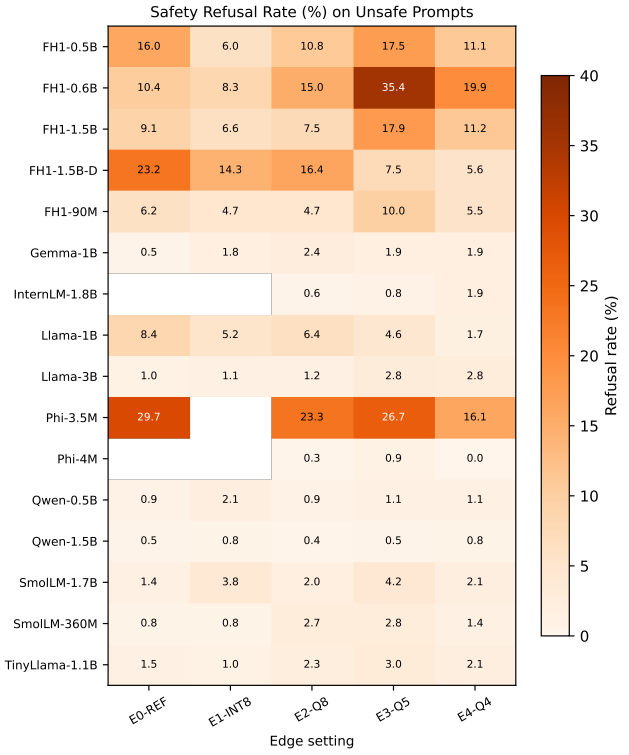

Figure 3: Safety refusal behaviour across model–setting pairs. Edge deployment does not induce a consistent safety trend: some models become more compliant under quantization, while others over-refuse benign prompts.

This result also cautions against treating refusal rate alone as alignment quality. A higher refusal rate may indicate safer behaviour on unsafe prompts, but it may also indicate over-refusal on benign prompts; conversely, a low refusal rate may reflect helpfulness on safe prompts or unsafe compliance. For this reason, we also measure over-refusal on safe control prompts (Appendix) and keep the classifier as a diagnostic tool rather than a final safety benchmark. The two rates must be read jointly: the low unsafe-prompt refusal we observe is a safety concern precisely because it is not accompanied by uniformly high helpfulness on safe prompts.

6 Discussion

What PEEL's lower bound means. PEEL should not be read as a replacement for upper-bound benchmarks. A model that performs poorly under P3 may still be useful when embedded in a product with a carefully engineered prompt, retrieval layer, and output parser. Likewise, a model that is robust under PEEL may still fail on tasks outside our English QA and instruction-following suites. The lower-bound inter-

pretation is more specific: if a model, runtime, and prompt tier combination fails here, then a practitioner should not assume that standard benchmark scores will transfer to a lightly engineered local deployment.

Task type and prompt degradation as primary moderators. Our corrected results show that *prompt quality*, not quantization level, is the dominant driver of SQuAD performance collapse. SQuAD v2 EM is stable at $\sim$21% across E0–E4 under expert prompts (P0), but collapses to $\sim$0% at P3 for every architecture and quantization setting. TruthfulQA-MC1 shows the opposite pattern: robust to both axes. At P0, accuracy changes by less than 1 pp across E0–E4; the small 4.8 pp drop in the all-tier setting average comes from the P1 casual tier losing its chat template on CPU (Section 5), not from quantization itself. This task-format interaction motivates separate lower-bound reporting rather than a single scalar headline metric.

The implication is that benchmark design should preserve task format as a first class factor. Multiple-choice selection, span extraction, binary judgment, and open-ended conversation do not degrade in the same way. A single average score would hide the most actionable lesson in our data: quantized models can remain usable when the answer space is constrained, while exact-reference tasks become fragile when the prompt no longer specifies the contract.

Two scoring artefacts and their lesson. Our audit identified two independent scoring bugs with a shared root cause: `llama.cpp` models at E2–E4 emit verbose, multi-line outputs that append EOS/stop-token strings and repeated prompt instructions after the actual answer. For *SQuAD v2*, surviving stop tokens block exact match and produce a spurious 0% EM at E2+; truncating to the first non-empty line and stripping EOS tokens recovers 21–31% EM, consistent with E0-REF. For *HaluEval*, the echoed instruction contains both "yes" and "no", so a full-text parser marks 50–62% of E2–E4 responses *ambiguous*; first-line parsing recovers accuracy from $\sim$24% to $\sim$47–51% (mechanics in Appendix A). Both are the same pitfall at different output levels: any `llama.cpp` evaluation that parses free text without first-line truncation will systematically underreport span-extraction *and* binary-classification accuracy. More broadly, local runtimes expose raw text that hosted chat APIs hide behind message-role formatting and stop-token logic, so benchmarks reusing hosted-model parsers on local outputs may measure interface mismatch rather than capability. Edge-LLM benchmarks should publish parsers with raw outputs and run a parser audit before ranking models.

Practical recommendations. For *practitioners*: prefer multiple-choice or classification outputs over extractive spans when running `llama.cpp` Q4/Q5; favour Falcon-H1 SSM-Hybrid models, which offer better instruction-following and prompt robustness than same-size dense Transformers at no extra memory cost; and re-test safety refusal after deployment conversion, since quantization does not produce a monotonic safety trend. For *benchmark builders*: report prompt-tier matched results rather than a single prompt-averaged mean, keep judge-scored MT-Bench separate from reference-scored tasks, and document missing cells explicitly—in an edge benchmark, tokenizer and chat-template failures are themselves part of the deployment burden.

Limitations. All datasets are English-only. P1–P3 prompt templates are handcrafted; a controlled user study validating tier realism is future work (see below). The E1$\rightarrow$E2 boundary confound has been partially isolated via a consumer-hardware replication (Appendix A), but a full GGUF-on-GPU ablation within the HPC cluster remains future work. Finally, the current safety analysis uses keyword-based refusal detection. It is useful for detecting instability across model–setting pairs, but it should not be treated as a substitute for human red teaming or policy-specific safety evaluation.

Ethical considerations. Our safety analysis shows that sub-2B models, once quantized for edge use, refuse only a small fraction of unsafe prompts. We report this as a cautionary finding, not a recipe: it underscores that deploying small local models on open user input without an added safety layer carries real risk. The evaluation uses existing public safety prompts and creates no new harmful capability; we release the refusal diagnostics precisely to encourage pre-deployment safety testing. Because PEEL is a lower-bound stress test, its scores should inform deployment risk assessment rather than be read as endorsements of any model for unsupervised use.

7 Conclusion

We introduced PEEL, a benchmark for simultaneously probing LLM performance along two real-world degradation axes: edge quantization and prompt quality. Evaluating 18 models across 1,184 reference-scored runs and 150 MT-Bench judge-scored runs reveals four principal findings: (1) task type is the strongest determinant of robustness—MC tasks degrade gracefully while extraction tasks collapse under informal prompts; (2) Falcon-H1-1.5B substantially outperforms same-size Transformers on instruction-following (8.20 vs. 4.06 MT-Bench at 1.5B scale) and remains stable from P0 to P3; (3) quantization-induced output artefacts can masquerade as capability collapse unless scoring strips stop tokens and prompt echoes; and (4) safety alignment is fragile at sub-2B scale: models refuse only 6.5% of unsafe prompts on average (max 35.4%) across model–setting pairs, and quantization produces no consistent safety trend. We release all prompts, evaluation scripts, and model outputs to support reproducibility and future lower-bound benchmarking research.

References

Anthony, R.; et al. 2024. An Empirical Study of Mamba-based Language Models.

Badawy, M.; et al. 2025. Sustainable LLM Inference for Edge AI: Evaluating Quantized LLMs on ARM-based Edge Platforms.

Cao, B.; Cai, D.; Zhang, Z.; et al. 2024. On the Worst Prompt Performance of Large Language Models.

Dettmers, T.; Lewis, M.; Belkada, Y.; and Zettlemoyer, L. 2022. LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale.

Dong, X.; et al. 2024. Hymba: A Hybrid-head Architecture for Small Language Models.

Frantar, E.; Ashkboos, S.; Hoefler, T.; and Alistarh, D. 2022. GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers.

Gerganov, G.; et al. 2023. GGUF: llama.cpp quantization format.

Gu, A.; and Dao, T. 2023. Mamba: Linear-Time Sequence Modeling with Selective State Spaces.

Haggenmiller, F.; et al. 2024. MobileQuant: Mobile-friendly Quantization for On-device Language Models.

Hendrycks, D.; et al. 2021. Measuring Massive Multitask Language Understanding. *ICLR*.

Kim, S.; Hooper, C.; Gholami, A.; et al. 2023. SqueezeLLM: Dense-and-Sparse Quantization.

Kwiatkowski, T.; Palomaki, J.; Redfield, O.; et al. 2019. Natural Questions: A Benchmark for Question Answering Research. In *TACL*.

Li, J.; Cheng, X.; Zhao, W. X.; et al. 2023. HaluEval: A Large-Scale Hallucination Evaluation Benchmark for Large Language Models.

Li, Y.; Liu, J.; Zhang, H.; et al. 2024. PalmBench: A Comprehensive Benchmark of Compressed Large Language Models on Mobile Platforms.

Liang, B.; He, L.; Luo, Z.; et al. 2023a. EdgeFM: Leveraging Foundation Model for Open-set Learning on the Edge.

Liang, P.; et al. 2023b. Holistic Evaluation of Language Models. In *TMLR*.

Lin, S.; Hilton, J.; and Evans, O. 2022. TruthfulQA: Measuring How Models Mimic Human Falsehoods. In *ACL*.

Ma, S.; Wang, H.; Ma, L.; et al. 2024. The Era of 1-bit LLMs: All Large Language Models are in 1.58 Bits.

Mizrahi, M.; Kaplan, G.; Malkin, D.; et al. 2024. State of What Art? A Call for Multi-Prompt LLM Evaluation.

Rajpurkar, P.; Jia, R.; and Liang, P. 2018. Know What You Don't Know: Unanswerable Questions for SQuAD. In *ACL*.

Sclar, M.; Choi, Y.; Tsvetkov, Y.; and Suhr, A. 2023. Quantifying Language Models' Sensitivity to Spurious Features in Prompt Design.

Srivastava, A.; et al. 2023. Beyond the Imitation Game: Quantifying and Extrapolating the Capabilities of Language Models. *TMLR*.

Tiiuae; et al. 2025. Falcon-H1: A Family of Hybrid-Head Language Models Redefining Efficiency and Performance.

Tseng, A.; Chee, J.; Sun, Q.; et al. 2024. QuIP#: Even Better LLM Quantization with Hadamard Incoherence and Lattice Codebooks.

Wang, A.; et al. 2019. SuperGLUE: A Stickier Benchmark for General-Purpose Language Understanding Systems. In *NeurIPS*.

Xiao, G.; et al. 2025. A Systematic Characterization of LLM Quantization for Efficient Inference.

Zheng, L.; Chiang, W.-L.; Sheng, Y.; et al. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *NeurIPS*.

Zhou, J.; Lu, T.; Mishra, S.; et al. 2023. Instruction-Following Evaluation for Large Language Models.

Zhu, K.; Wang, J.; Zhou, J.; et al. 2023. PromptBench: Towards Evaluating the Robustness of Large Language Models on Adversarial Prompts.

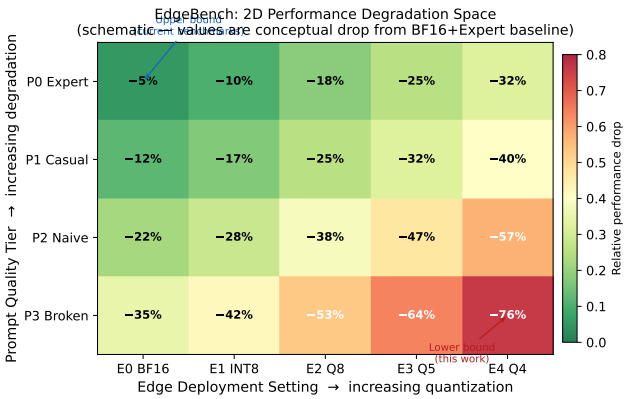

Figure 4: Schematic of the PEEL 2D evaluation space. Each cell is one experimental condition; colour encodes schematic relative performance drop from the top-left (expert prompt, full-precision) baseline.

Figure 2: TruthfulQA-MC1 across Edge Settings × Prompt Tiers

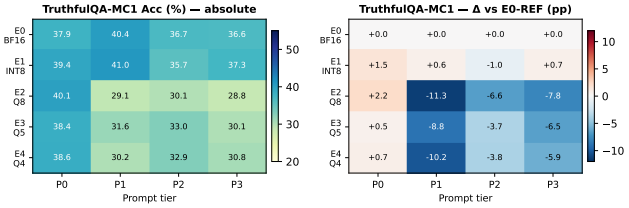

Figure 5: TruthfulQA-MC1 mean accuracy (%) across the full 4×5 evaluation grid (left) and delta relative to E0-REF (right). Chance baseline: 25.0%. Both axes show modest effects—no systematic collapse.

A Appendix: Additional Experimental Details

Full Result Grids and Throughput

The main text presents SQuAD v2 as a heatmap (Figure 1) and summarises the other reference-scored grids in prose. For completeness we give the full 4×5 grids here, together with the TruthfulQA heatmap, decode throughput, and a consumer-hardware replication of the corrected SQuAD scores.

Latency vs. accuracy trade-off. Table 8 reports mean decode throughput by setting. E0-REF (BF16 GPU) achieves the highest throughput at 116.7 chars/s. Counterintuitively, E1-INT8 (GPU int8) is slower at 35.3 chars/s due to `bitsandbytes` dequantization overhead. E2–E4 CPU settings achieve 23.8–31.7 chars/s without any GPU—practically sufficient for non-real-time applications. E4-Q4 reduces memory footprint by $\sim 4\times$ relative to E0-REF BF16 while retaining $\sim$87% of TruthfulQA accuracy at P0.

Consumer-hardware replication. To validate the corrected scores, we ran a supplementary local evaluation on an Intel i7-13700KF workstation (8 threads, CPU-only `llama.cpp` b8937) covering Falcon-H1 (our primary SSM-Hybrid family) and Qwen2.5 (Transformer reference). Using the fixed scorer, **Falcon-H1-0.5B** achieved 14–16% EM (Q8_0/Q4) and **Falcon-H1-1.5B** achieved 22% EM (Q4)—consistent with the corrected HPC scores (31%/30.5% for the same models at E2–E4, with differences attributable to output verbosity). Qwen2.5-0.5B similarly reached 20–26% EM locally vs. 24–30% corrected HPC. Throughput on the consumer CPU was 29–94 chars/s for Falcon-H1 (vs. 24–32 chars/s on HPC), with Q4 faster than Q8 due to memory bandwidth.

Table 6: SQuAD v2 Exact Match (%) in the 4×5 grid. P0 is stable across all quantization levels; P3 is universally zero.

Setting	P0	P1	P2	P3
E0-REF	19.8	7.9	0.1	0.0
E1-INT8	20.5	5.8	0.0	0.0
E2-Q8	22.0	12.4	0.2	0.0
E3-Q5	21.8	10.9	0.1	0.0
E4-Q4	20.1	10.4	0.1	0.0

Table 7: NQ-Open Exact Match (%) in the 4×5 grid. E2–E4 are uniformly zero: the E1→E2 boundary marks a complete factual recall collapse for sub-2B models on llama.cpp without a chat template.

Setting	P0	P1	P2	P3
E0-REF	1.3	0.0	0.0	0.0
E1-INT8	1.7	0.0	0.0	0.0
E2-Q8	0.0	0.0	0.0	0.0
E3-Q5	0.0	0.0	0.0	0.0
E4-Q4	0.0	0.0	0.0	0.0

Prompt Tier Operationalization

The prompt tiers in PEEL are intended as a controlled diagnostic axis, not as a calibrated psychometric model of user skill. P0 preserves the conventional benchmark prompt with explicit task framing and output constraints. P1 removes some technical phrasing while keeping grammatical task intent. P2 gives the model the minimum content needed to infer the task. P3 introduces informal phrasing, abbreviations, and weak formatting. The current paper uses one template per task and tier; future work should sample multiple templates per tier or collect natural prompts from users.

Run Coverage

Table 10 summarizes the coverage used by the main analysis. Reference-scored tasks use 200 examples per run after filtering; MT-Bench uses 80 questions and is therefore reported separately from the 100-example filter used for QA/classification aggregates.

MT-Bench Coverage and Exclusions

MT-Bench is evaluated only at P0 and P3 because it is an open-ended instruction-following benchmark rather than a reference-scored QA task. This means MT-Bench is not included in the P0/P1/P2/P3 matched heatmaps. Instead,

Table 8: Mean decode throughput (chars/s) by edge setting.

Setting	Mean	Min	Max
E0-REF (BF16, GPU)	116.7	3.1	327.8
E1-INT8 (int8, GPU)	35.3	0.9	89.8
E2-Q8 (Q8, CPU)	23.8	1.4	180.0
E3-Q5 (Q5_K_M, CPU)	27.0	1.1	193.7
E4-Q4 (Q4_K_M, CPU)	31.7	1.2	232.2

the paper reports matched P3–P0 deltas by model, family, and setting. Falcon-E is excluded because it runs through a native BitNet inference path rather than the HuggingFace/GGUF path used for MT-Bench generation. InternLM and Phi have partial setting coverage because of tokenizer and chat-template failures.

Scoring Artefact Corrections

Two scoring artefacts were discovered during result auditing. First, `llama.cpp` generations for SQuAD often included stop tokens, prompt echoes, or trailing explanatory text after the answer. Exact-match scoring therefore underreported span accuracy. The corrected scorer strips known EOS tokens and truncates to the first non-empty output line before applying standard SQuAD normalization. Second, HaluEval generations sometimes repeated the instruction text after the answer; because the instruction contains both "yes" and "no", a full-text parser marked many outputs as ambiguous. The corrected parser again uses the first non-empty line.

These corrections are conservative: they do not search the entire response for a favorable answer, and they do not change the gold labels. They only remove post-answer text that would not be shown as the answer in a deployed QA or classification interface.

Safety Classifier Caveats

The safety analysis is based on a keyword/refusal classifier with three labels: REFUSE, COMPLY, and AMBIGUOUS. The current draft reports refusal rates rather than attack success rates because ambiguous responses are common in small edge models. For unsafe prompts, a mean refusal rate of 6.5% (max 35.4% across model–setting pairs) means the large majority of unsafe requests are either complied with or ambiguous. We additionally report over-refusal on safe control prompts as a diagnostic. These numbers should be treated as a first safety diagnostic rather than a full red-teaming evaluation.

Failure Case Taxonomy

During result auditing, we grouped failures into four recurring types. This taxonomy is useful for interpreting PEEL because not all low scores imply the same kind of weakness.

This taxonomy guided the interpretation of the main results. SQuAD P3 is mainly a task-contract failure; NQ-Open at E2–E4 combines knowledge limits with runtime-interface failure; HaluEval's initial low scores were primarily a parser and runtime-interface issue; and safety variation reflects policy instability rather than a monotonic quantization effect.

Why MT-Bench Is Not in the 4-Tier Heatmap

MT-Bench is a classic eight-category, open-ended, judge-scored benchmark. Its unit of evaluation is an 80-question conversation set rather than a 200-example reference-scored sample. We therefore do not mix it into the P0/P1/P2/P3 reference-task heatmaps. Instead, we use MT-Bench to test a different question: whether informal prompting changes open-ended instruction-following quality and whether model-family rankings are stable under deployment conversion. The main paper reports setting-level and family-level P0/P3 deltas; category-level radar plots are included in the released analysis notebook.

Interpreting Missing Cells

Missing cells in PEEL are not silently imputed. They arise from three sources: (1) model families that require a different runtime, such as Falcon-E with `bitnet.cpp`; (2) tokenizer or chat-template incompatibilities that prevent reliable generation for a task/setting; and (3) optional baseline models with partial coverage. We report aggregates over available cells and avoid claiming a comparison when the matched model–setting–prompt set is incomplete. For prompt-sensitivity claims, we use matched P0/P3 pairs wherever possible.

Reproducibility Notes

The released repository contains three layers of artefacts: raw generations, per-run score files, and aggregate CSV summaries. The intended workflow is: (1) run generation jobs with a model key, setting key, dataset key, and prompt tier; (2) score raw outputs with the task-specific evaluator; (3) aggregate per-run metrics into `results/summary.csv`; and (4) derive paper tables and figures from the summary plus MT-Bench `scores.jsonl` files. This separation is deliberate. It allows parser fixes, such as the SQuAD EOS correction and HaluEval first-line correction, to be applied without rerunning model inference.

For a new model, the minimal experiment matrix is P0/P3 for SQuAD v2, TruthfulQA-MC1, NQ-Open, and MT-Bench under E0 plus the intended edge setting. The full PEEL matrix then expands to all four prompt tiers for reference-scored tasks and all five settings for deployment analysis. We recommend running smoke tests with a small example limit before launching the full 200-example runs, because runtime failures are common in the edge setting and should be discovered before queueing large jobs.

Threats to Validity

Prompt realism. P1–P3 are hand-authored diagnostic prompts. They represent plausible levels of prompt specificity, but they are not drawn from a user study. The claims should therefore be read as robustness-to-controlled-prompt-degradation claims, not as population-level claims about all novice users.

Hardware realism. E2–E4 are CPU-only GGUF deployments with constrained thread and memory settings. They approximate a local edge environment, but they do not cover

Table 9: Prompt-tier definitions used in the reference-scored tasks.

Tier	Intended user type	Operational rule
P0	Benchmark/expert	Explicit instruction, labelled fields, and output-format constraint.
P1	Casual but clear	Natural wording with task intent preserved; fewer labels and weaker format constraint.
P2	Naive/minimal	Question and context/options only; no explicit task description.
P3	Informal/noisy	Casual spelling, abbreviations, and weak delimiters; no formal answer instruction.

Table 10: Run coverage used in the current draft.

Component	Runs	Notes
Reference-scored tasks	1,184	200 examples per run after filtering
MT-Bench	150	80 questions; P0/P3 only
IFEval (Zhou et al. 2023)	45	Reported in released artifacts, not central in main text
Safety	75 model–setting pairs	Keyword/refusal classifier over safe and unsafe prompts

Table 11: MT-Bench setting-level means.

Setting	P0	P3
E0-REF	5.00	4.87
E1-INT8	5.12	4.86
E2-Q8	5.02	4.92
E3-Q5	4.79	4.58
E4-Q4	4.64	4.54

phones, NPUs, mobile GPUs, or vendor-specific acceleration. We removed phone-device comparisons from the current paper because no phone measurements were collected.

Judge reliability. MT-Bench relies on GPT-4o-mini as an automatic judge. The judge is useful for relative instruction-following comparisons, but it introduces model-specific judgment noise. For this reason, MT-Bench conclusions are reported separately from reference-scored QA and classification tasks.

Safety scope. The safety suite measures refusal/compliance patterns with a keyword classifier. It is appropriate for detecting deployment instability, but insufficient for policy compliance claims. A future version should include human annotation or a stronger policy judge.

Table 12: Failure categories observed during PEEL auditing.

Failure type	Observable symptom	Main affected tasks
Task-contract failure	Model answers conversationally, explains, or emits extra text instead of the required span/letter/label.	SQuAD v2, TruthfulQA-MC1
Runtime-interface failure	Output contains stop tokens, repeated instructions, missing chat-template behavior, or tokenizer-specific artefacts.	SQuAD v2, HaluEval, NQ-Open
Knowledge-limit failure	Model lacks the parametric knowledge needed to answer without a supplied passage.	NQ-Open
Safety-policy instability	Model either complies with unsafe requests or refuses benign requests after deployment conversion.	Safety suite